

Everything you need to say, and everything you might get asked

A full talking script for each slide of Krypisis_Presentation.pptx, plus the questions most likely to come up and prepared answers — including the awkward ones: why accuracy is 100%, and why our research question came out “no”. Every number here is generated from the project's results files, so it always matches the slides.

How to use this

Read once, straight through, the night before. On presentation day, skim the **Script** boxes right before you go up — they're written to be said almost as-is. The **Q&A** boxes are the questions most likely to come from an evaluator; know the shape of each answer rather than memorizing it.

- Know which teammate is presenting which slide before you walk in.
- Have the GitHub repo open in a browser tab, in case anyone asks to see real code or results.
- Read the “Hard Questions” chapter twice — especially the 100% question and the “no” answer.

0 Title Slide

ON SCREEN

On screen: project title, subtitle, team names.

SCRIPT

“Good [morning/afternoon]. We're presenting Krypsis — a Federated Learning-based Network Intrusion Detection System with a custom communication protocol. I'm [name], and with me are [names] — we'll each cover a section.”

What does “Krypsis” mean?

Greek for “hiding” or “concealment” — fitting, since the system never exposes raw traffic, only model updates.

1 Introduction

Why this project, and what it is

ON SCREEN

On screen: why we chose it (privacy problem with centralized NIDS) · what it is (NIDS + Federated Learning across 10 routers + our own protocol).

SCRIPT

“An intrusion detection system needs to learn from a lot of varied network traffic. But no organization wants to hand its traffic logs to an outside server — it reveals internal structure, and privacy rules often forbid it.

Federated Learning flips the flow: instead of moving data to the model, we send the model to the data. Each router trains locally and only sends back what it learned. On top of that we built our own protocol for sending those updates, which checks every update for tampering and for signs of a malicious client before it is used.”

Isn't Federated Learning already well known?

Yes, and we say so. Our contribution is the protocol layer on top — integrity checking and cheap anomaly filtering of updates — and the experiments testing it under attack.

What is a “client” in your system?

A router. Our dataset, FLNET2023, was recorded at 10 routers of an emulated network, so each router is one federated client holding only the traffic it saw.

2 Application

Where this matters, and what we did

ON SCREEN

On screen: real-world use cases · what we built (FLNET2023, 10 router clients, FedAvg, protocol attacked and repeated on NSL-KDD).

SCRIPT

“This matters anywhere organizations want to cooperate on security without exposing their networks — hospitals, banks, or IoT networks where every gateway learns locally, which is the fog computing idea from our syllabus.

We used FLNET2023, a 2023 dataset built specifically for federated intrusion detection: traffic from 10 routers, each seeing a different mix of attacks — some routers see almost only normal traffic, others mostly attacks. We trained one shared model across all 10, then attacked our own system — poisoned updates, tampered messages, backdoors — and repeated the experiments on the classic NSL-KDD dataset to check the results hold.”

Why switch from NSL-KDD to FLNET2023?

NSL-KDD has no notion of clients, so the uneven split across clients had to be faked. FLNET2023's clients are real routers with genuinely different traffic — the realistic setting our research question is about. We kept NSL-KDD as a second dataset.

3 Tools Used

The toolchain behind it

ON SCREEN

On screen: Python, TensorFlow/Keras, scikit-learn, pandas/NumPy, FLNET2023 + NSL-KDD, Git/GitHub.

SCRIPT

“Our tools are a machine-learning stack, not a network simulator — intentionally. Python, TensorFlow and Keras for the neural network, scikit-learn for preprocessing and metrics, pandas and NumPy for the data, and Python's standard hmac and hashlib for the protocol's integrity tag. Everything is on GitHub with the full history.”

Why no network simulator like Packet Tracer or Mininet?

Our contribution is at the learning and protocol-design level. FLNET2023 itself was recorded in an emulated network (the CORE emulator), so the network side is already in the data. Running the protocol over Mininet would be a natural next step.

What is FLNET2023, exactly?

A public dataset (Kumar et al., MILCOM 2023) of network flows recorded at 10 routers, with flow features extracted by CICFlowMeter — durations, packet counts and sizes, flags, timing — labelled as normal or one of 10 attack types.

4 Methodology

How it was built, step by step

ON SCREEN

On screen: 4-step pipeline · the FedAvg loop over the Krypsis protocol.

SCRIPT

“Four stages. First, preprocessing: 66 flow features per connection, and we deliberately remove IP addresses, ports and timestamps so the model learns what traffic looks like, not who sent it. Second, one client per router. Third, the model: a neural network with layers of 256, 128 and 64 neurons.

Fourth, federated training. Each round the server sends the model to the 10 routers, each trains it on its own traffic, and sends back only the updated weights inside a Krypsis message. The server verifies each message, filters suspicious ones, and averages the rest.”

Why remove IP addresses and timestamps?

In the emulated network attackers use fixed addresses. Keeping them would let the model memorize “this IP is the attacker” — useless on a real network.

Why weight the averaging by data size?

A router with 140,000 flows has seen far more evidence than one with 20,000. That is standard FedAvg. Our improved protocol caps any single router at 15%, for a reason we explain in the results.

5 Status & Results

What's done, and the real numbers

ON SCREEN

On screen: 100.0% centralized · 99.93% FedAvg over routers · 99.999% IID · 105/105 tampered messages rejected · phase table, all complete.

SCRIPT

“Trained normally, on all data in one place, our model reaches 100.0% on the test set. Trained federated across the 10 routers it reaches 99.93% — so keeping data private costs almost nothing.

And in our protocol experiments, every single tampered message — 105 out of 105 — was caught by the integrity check before it could affect the model.”

“Because the binary task is so easy, we also trained a model that names the exact traffic type out of 11: it reaches 68.9% accuracy federated, macro-F1 0.389.”

Isn't 100% accuracy suspicious?

Yes — so we checked it rather than trusting it. See “Hard Questions”: the dataset is simply very easy to separate; one feature alone scores 97.5%.

Why is federated slightly lower than centralized?

Each router trains only on its own traffic, and most routers never see most attack types — averaging can't fully make up for that. Every remaining error in the router split is a missed attack.

6 The Krypsis Protocol

What one update message looks like

ON SCREEN

On screen: message layout (length, header + fingerprint, payload, HMAC tag) · +1,559 B overhead (0.67%) · the server's 5 checks.

SCRIPT

“Every update a router sends has four parts: a length field, a header with a fingerprint of the update — its size, size per layer, and a 64-number sketch of its direction — the update itself, and a 32-byte HMAC tag, a cryptographic checksum using a key only that router and the server share.

The server first checks the tag: if even one byte changed in transit, it's rejected. Then it recomputes the fingerprint from the update itself, so a client can't lie about it. Then it scores the update against the other routers in the same round — is it unusually large, does it point a different way, did it suddenly change from last round? Suspicious updates get a second, more careful check. Finally the accepted updates are averaged, with no router above 15%.

All of this adds only 0.67% to each message — sending the same update as plain JSON over HTTP would be 5.6 times larger.”

Why compare each update to the others in the same round?

We first compared against fixed thresholds, and it flagged 65–96% of honest updates: once training settles, every update becomes small and noisy. Scoring relative to the round fixed it — documented as a negative result.

What is the 64-number sketch?

A random projection of the update — like a compressed signature. Two updates pointing the same way have similar sketches, so the server can compare directions cheaply.

7 Protocol Results

Does it stop the attacks?

ON SCREEN

On screen: table of no defense vs protocol v1 vs v2, both datasets, label flip + backdoor · Mondrian result line.

SCRIPT

“On NSL-KDD the protocol clearly works. Under label flipping, accuracy without a defense falls to 68.8%; with our protocol it stays at 79.2%. The backdoor attack succeeds 34.1% of the time without a defense, and 8.7% with it. Both are statistically significant over 10 seeds.

On FLNET2023 it helps too — 92.4% rising to 99.4% — but with only 5 seeds that isn't significant, and the backdoor attack failed there even without any defense, so it tells us nothing about the protocol.

Our research question was whether per-group, Mondrian thresholds beat one global threshold. Over every experiment: no significant difference. That's a real answer, and we explain why.”

What is the difference between v1 and v2?

v1 drops every flagged update. v2 adds the second check, which rescues honest updates that were wrongly flagged, and the 15% weight cap. On NSL-KDD v2 cut wrongly excluded honest updates from 5.4% to 1.1%, and rescued no attackers.

Why did v2 do worse than v1 on FLNET2023?

Because of the weight cap, in one seed. When the large router 10 is honest, capping it hands part of its weight to the others — including the attackers. A fixed cap is not a free fix.

8 Learning

How this connects to our syllabus

ON SCREEN

On screen: three cards mapping the project to Units 1, 2 and 3.

SCRIPT

“Every FLNET2023 record is a network flow captured at a router — ports, packet counts and sizes, TCP flags, timing — straight from Units 1 and 2. Our clients literally are routers, the network edge.

Designing the Krypsis message format is application-layer protocol design, the same exercise as comparing HTTP with a custom protocol — and we measured exactly that comparison. And Federated Learning is fog computing from Unit 3: process at the edge, send only summaries up.”

Give one dataset feature tied to a specific layer.

dst_port — the destination port — is a Transport-layer concept identifying the application; the TCP flag counts (SYN, ACK, FIN) are Transport layer; packet lengths come from the Network layer.

9 Findings

What we learned — including what didn't work

ON SCREEN

On screen: 4 findings — binary task too easy · first protocol flagged honest clients · Mondrian not significantly better · fixed weight cap has a downside.

SCRIPT

“We want to end on what we learned, including what didn't work, because that's where most of the learning was. The dataset turned out to be too easy for the binary task, so we checked it and added multi-class detection. Our first protocol design flagged almost every honest update, so we found why and fixed it. Our research question came out negative. And one of our own fixes, the weight cap, has a real downside we only found by testing it.”

Isn't a negative result a failure?

No — it's an answer. We tested a clear hypothesis with repeated seeds and significance tests. Reporting it honestly is more useful than tuning until something looks good.

The hard questions

THE ONE TO ACTUALLY PREPARE FOR

“100% accuracy? That has to be a mistake — or cheating.”

Agree that it looks suspicious — then show you checked: “We thought so too, so we tested it. IP addresses, ports and timestamps are removed, so the model can't just memorize the attacker. Then we tried the simplest possible models: a single threshold on one feature, the forward packet count, already scores 97.5%, and a decision tree with three questions scores 99.99%.

So it isn't our model being clever — the dataset is emulated, and normal traffic and attack tools produce very different flows. That's why we don't rely on binary accuracy to judge anything: our protocol results are about how attacks change accuracy, and we added multi-class detection, which is harder.”

THE SECOND ONE

“Your research question came out ‘no’ — so what did the project achieve?”

“Three things that work: an integrity check that caught every tampered message, an anomaly filter that significantly improves accuracy under attack on NSL-KDD, and a second check that cut wrongly rejected honest updates about five times without letting attackers back in.

The Mondrian question is answered, not failed: Mondrian caught 57.6% vs 53.3% of poisoned updates — not significantly different. The likely reason is data: with 10 clients and 5 calibration rounds, each cluster has too few examples for a reliable threshold. More clients or longer calibration is the obvious next experiment.”

What is the single most novel thing about this project?

Fusing security into the update protocol itself: the integrity tag and a cheap fingerprint check happen before aggregation, and we tested — with real router clients — whether that cheap check treats honest, unusual clients unfairly.

Why only 5 seeds on FLNET2023?

Time. Each FLNET2023 run trains 10 clients on 564,000 flows for 12 rounds; 5 seeds × 2 attacks × 4 policies is already 40 runs. NSL-KDD is smaller, so it got 10 seeds — and that's where the significant results are.

What would you do differently if you started over?

Check how easy the dataset is before building on it, and plan experiments with enough seeds from the start — several of our FLNET2023 differences might become significant with more runs.

Rapid-fire glossary

If asked to define something on the spot, these are the one-liners.

NIDS

Software that watches network traffic and classifies it as normal or an attack.

Federated Learning

Training one shared model across many data owners, without any of them sharing their raw data.

FedAvg

The algorithm that combines clients' trained models into one, weighted by how much data each client had.

IID / non-IID

Whether every client's data looks statistically similar (IID) or genuinely different (non-IID) — FLNET2023's routers are naturally non-IID.

HMAC

A checksum computed with a secret key; anyone changing the message can't produce a matching tag without the key.

Fingerprint

Our small summary of an update — sizes plus a 64-number sketch — used to spot suspicious updates cheaply.

Label flipping

An attack where a malicious client trains on deliberately wrong labels.

Backdoor

An attack that teaches the model one hidden mistake — e.g. “let this attack type through” — while staying accurate otherwise.

Mondrian calibration

A separate detection threshold per group of similar clients instead of one global threshold.

Trimmed mean

Averaging after discarding the most extreme values — robust to a few bad inputs.

p-value

How likely a difference this big would appear by chance; below 0.05 we call it statistically significant.